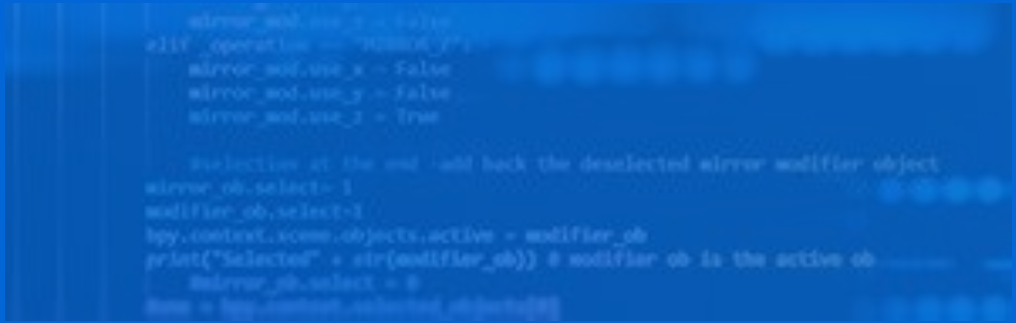
UT 4: DISEÑO FÍSICO. LENGUAJE DE DEFINICIÓN DE DATOS (DDL)



Lenguaje de Definición de Datos (DDL)

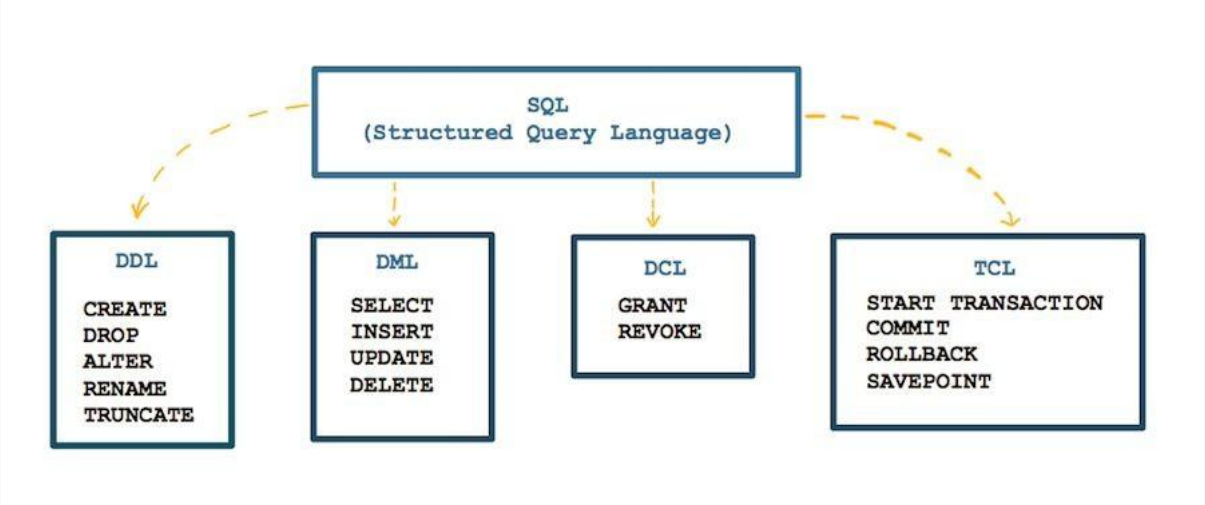
El lenguaje DDL de SQL

El **DDL** (*Data Definition Language*) o Lenguaje de Definición de Datos es la parte de SQL dedicada a la definición de los datos. Las sentencias DDL son las siguientes:

- CREATE: se utiliza para crear objetos como bases de datos, tablas, vistas, índices, *triggers* y procedimientos almacenados
- DROP: se utiliza para eliminar los objetos de la base de datos.
- ALTER : se utiliza para modificar los objetos de la base de datos.
- SHOW : se utiliza para consultar los objetos de la base de datos.

Otras sentencias de utilidad que usaremos en esta unidad son:

- **USE:** se utiliza para indicar la base de datos con la que queremos trabajar.
- **DESCRIBE:** se utiliza para mostrar información sobre la estructura de una tabla.



Lenguaje de Definición de Datos (DDL)

Sintaxis

La sintaxis de SQL se define mediante instrucciones que tiene diferentes parámetros, unos opcionales, otros con diferentes valores, etc.. La nomenclatura que vamos a emplear al explicar la sintaxis de cada instrucción será la siguiente:

Símbolo	Descripción
MAYÚSCULAS	Palabras reservadas (*keywords*) de SQL
minúsculas	variable que hay que sustituir por un elemento concreto
[ ]	opcional
|	separa opciones alternativas
[|]	se elige uno de los valores
{ }	obliga a elegir uno de los valores
...	número variable de datos

Recuerda que todas las instrucciones terminan en ; y que vamos a dejar en MAYÚSCULAS las keywords para facilitar la lectura, aunque no es obligatorio escribirlas así.

Lenguaje de Definición de Datos

Manipulación de Bases de Datos

Crear una base de datos

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] nombre_base_datos;
```

- DATABASE y SCHEMA son sinónimos.
- IF NOT EXISTS crea la base de datos sólo si no existe una base de datos con el mismo nombre.

Ejemplos:

Si no especificamos el set de caracteres en la creación de la base de datos, se usará latin1 por defecto.

```
CREATE DATABASE nombre_base_datos;
```

Las bases de datos que vamos a crear usarán el set de caracteres utf8 o utf8mb4.

```
CREATE DATABASE nombre_base_datos CHARACTER SET utf8;
```

El cotejamiento, es el criterio que vamos a utilizar para ordenar las cadenas de caracteres de la base de datos. Si no especificamos ninguno se usará el que tenga asignado por defecto el set de caracteres escogido. Por ejemplo, para el set de caracteres utf8 se usa utf8_general_ci. El siguiente ejemplo muestra cómo podemos especificar el cotejamiento queremos de forma explícita:

```
CREATE DATABASE nombre_base_datos CHARACTER SET utf8 COLLATE utf8_general_ci;
```

Manipulación de Bases de Datos

Crear una base de datos. Codificación de caracteres

Unicode es un set de caracteres universal, un estándar en el que se definen todos los caracteres necesarios para la escritura de la mayoría de los idiomas hablados en la actualidad. El estándar Unicode describe las propiedades y algoritmos necesarios para trabajar con los caracteres Unicode y este estándar es gestionado por el consorcio Unicode.

Los formatos de codificación que se pueden usar con Unicode se denominan UTF-8, UTF-16 y UTF-32.

- UTF-8 utiliza 1 byte para representar caracteres en el set ASCII, 2 bytes para caracteres en otros bloques alfabéticos y 3 bytes para el resto del BMP (Basic Multilingual Plane), que incluye la mayoría de los caracteres utilizados frecuentemente. Para los caracteres complementarios se utilizan 4 bytes.
- UTF-16 utiliza 2 bytes para cualquier carácter en el BMP y 4 bytes para los caracteres complementarios.
- UTF-32 emplea 4 bytes para todos los caracteres.

Crear una base de datos.: utf8 y utf8mb4 en MySQL

En MySQL el set de caracteres utf8 utiliza un máximo de 3 bytes por carácter y contiene sólo los caracteres del BMP (Basic Multilingual Plane). Según el estándar Unicode, el formato de codificación utf8 permite representar caracteres desde 1 hasta 4 bytes, esto quiere decir que el set de caracteres utf8 de MySQL no permite almacenar caracteres Unicode con 4 bytes.

Este problema se soluciona a partir de MySQL 5.5.3, cuando se añadió el set de caracteres utf8mb4 que permite utilizar hasta 4 bytes por carácter.

Por ejemplo, en MySQL los caracteres Emoji Unicode no se podrían representar con utf8, habría que utilizar utf8mb4:

Emoji	Unicode	Bytes (UTF-8)
😃	U+1F603	\xF0\x9F\x98\x83
🙈	U+1F648	\xF0\x9F\x99\x88
👾	U+1F47E	\xF0\x9F\x91\xBE

En la documentación oficial de MySQL informan que en las próximas versiones de de MySQL se espera solucionar este problema haciendo que **utf8** sea un alias de **utf8mb4**. Hasta que esto no ocurra, se recomienda utilizar **utf8mb4**

Manipulación de Bases de Datos

Crear una base de datos. CHARACTER SET y COLLATE

CHARACTER SET: Especifica el set de caracteres que vamos a utilizar en la base de datos.

COLLATE: Especifica el tipo de cotejamiento que vamos a utilizar en la base de datos. Indica el criterio que vamos a seguir para ordenar las cadenas de caracteres.

Para ver cuáles son los sets de caracteres que tenemos disponibles podemos usar la siguiente sentencia:

```
SHOW CHARACTER SET;
```

Para consultar qué tipos de cotejamiento hay disponibles podemos usar:

```
SHOW COLLATION;
```

Si queremos hacer una consulta más específica sobre los tipos de cotejamiento que podemos usar con utf8:

```
SHOW COLLATION LIKE 'utf8%';
```

Manipulación de Bases de Datos

Crear una base de datos. CHARACTER SET y COLLATE

El cotejamiento puede ser:

- case-sensitive (_cs): Los caracteres a y A son diferentes.
- case-insensitive (_ci): Los caracteres a y A son iguales.
- binary (_bin): Dos caracteres son iguales si los valores de su representación numérica son iguales.

Para consultar qué set de caracteres y qué cotejamiento está utilizando una determinada base de datos podemos consultar el valor de las variables character_set_database y collation_database.

En primer lugar seleccionamos la base de datos con la que vamos a trabajar.

```
USE database;
```

Y una vez seleccionada, consultamos el valor de las variables `character_set_database` y `collation_database`.

```
SELECT @@character_set_database, @@collation_database;
```

Lenguaje de Definición de Datos

Manipulación de Bases de Datos

Crear una base de datos.

Ejemplo de cómo afecta el cotejamiento al ordenar una tabla

Suponemos que tenemos una tabla que contiene cadenas de caracteres codificadas en latin1.

```
mysql> CREATE TABLE t (c CHAR(3) CHARACTER SET latin1);
mysql> INSERT INTO t (c) VALUES('AAA'),('bbb'),('aaa'),('BBB');
mysql> SELECT c FROM t;
+------+
| c  |
+------+
| AAA |
| bbb |
| aaa |
| BBB |
+------+
```

Cotejamiento case-sensitive (los caracteres a y A son diferentes).

```
mysql> SELECT c FROM t ORDER BY c COLLATE latin1_general_cs;
+------+
| c  |
+------+
| AAA |
| aaa |
| BBB |
| bbb |
+------+
```

Cotejamiento case-insensitive (los caracteres a y A son iguales).

```
mysql> SELECT c FROM t ORDER BY c COLLATE latin1_swedish_ci;
+------+
| c     |
+------+
| AAA |
| aaa |
| bbb |
| BBB |
+------+
```

Cotejamiento binary (dos caracteres son iguales si los valores de su representación numérica son iguales).

```
mysql> SELECT c FROM t ORDER BY c COLLATE latin1_bin;
+------+
| c    |
+------+
| AAA  |
| BBB  |
| aaa  |
| bbb  |
+------+
```

Lenguaje de Definición de Datos

Manipulación de Bases de Datos

Eliminar una base de datos.

```
DROP {DATABASE | SCHEMA} [IF EXISTS] nombre_base_datos;
```

- DATABASE y SCHEMA son sinónimos.
- IF EXISTS elimina la base de datos sólo si ya existe.

Ejemplo:

```
DROP DATABASE nombre_base_datos;
```

Modificar una base de datos.

```
ALTER {DATABASE | SCHEMA} [nombre_base_datos]
    alter_specification [, alter_especification] ...
```

Ejemplo:

```
ALTER DATABASE nombre_base_datos CHARACTER SET utf8;
```

Lenguaje de Definición de Datos

Manipulación de Bases de Datos

Consultar el listado de bases de datos disponibles

```
SHOW DATABASES;
```

Se utiliza para indicar la base de datos con la que queremos trabajar.

Seleccionar una base de datos

```
USE nombre_base_datos;
```

Se utiliza para indicar la base de datos con la que queremos trabajar.

Mostrar la sentencia SQL de creación de una base de datos

```
SHOW CREATE DATABASE nombre_base_datos;
```

Se puede utilizar para visualizar la sentencia SQL que sería necesaria ejecutar para crear la base de datos que le estamos indicando como parámetro.

Lenguaje de Definición de Datos

Manipulación de Tablas

Crear una tabla

A continuación se muestra una versión simplificada de la sintaxis necesaria para la creación de una tabla en MySQL.

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
    (create_definition,...)
    [table_options]

create_definition:
    col_name column_definition
  | [CONSTRAINT [symbol]] PRIMARY KEY (index_col_name,...)
  | [CONSTRAINT [symbol]] FOREIGN KEY (index_col_name,...) reference_definition
  | CHECK (expr)

column_definition:
    data_type [NOT NULL | NULL] [DEFAULT default_value]
      [AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]

reference_definition:
    REFERENCES tbl_name (index_col_name,...) [ON
      DELETE reference_option]
      [ON UPDATE reference_option]
reference_option:
    RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT

table_options:
    table_option [[,] table_option] ...

table_option: AUTO_INCREMENT [=]
    value
  | [DEFAULT] CHARACTER SET [=] charset_name
  | [DEFAULT] COLLATE [=] collation_name
      | ENGINE [=] engine_name
```

Manipulación de Tablas

Crear una tabla - Restricciones sobre las columnas de la tabla

Podemos aplicar las siguientes restricciones sobre las columnas de la tabla:

- **NOT NULL o NULL**: Indica si la columna permite almacenar valores nulos o no.
- **DEFAULT:** Permite indicar un valor inicial por defecto si no especificamos ninguno en la inserción.
- **AUTO_INCREMENT:** Sirve para indicar que es una columna autonumérica. Su valor se incrementa automáticamente en cada inserción de una fila. Sólo se utiliza en campos de tipo entero.
- **UNIQUE KEY**: Indica que el valor de la columna es único y no pueden aparecer dos valores iguales en la misma columna.
- **PRIMARY KEY:** Para indicar que una columna o varias son clave primaria.
- **CHECK:** Nos permite realizar restricciones sobre una columna. En las versiones previas a MySQL 8.0 estas restricciones no se aplicaban, sólo se parseaba la sintaxis pero eran ignoradas por el sistema gestor de base de datos. A partir de la versión de MySQL 8.0 ya sí se aplican las restricciones definidas con CHECK.

Lenguaje de Definición de Datos

Manipulación de Tablas

Crear una tabla - Restricciones sobre las columnas de la tabla

Ejemplo 1

```sql
DROP DATABASE IF EXISTS proveedores;
CREATE DATABASE proveedores CHARSET utf8mb4;
USE proveedores;

CREATE TABLE categoria (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
);

CREATE TABLE pieza (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  color VARCHAR(50) NOT NULL,
  precio DECIMAL(7,2) NOT NULL CHECK (precio > 0),
  id_categoria INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_categoria) REFERENCES categoria(id)
);
```

Lenguaje de Definición de Datos

Ejemplo 2

```sql
DROP DATABASE IF EXISTS agencia;
CREATE DATABASE agencia CHARSET utf8mb4;
USE agencia;

CREATE TABLE turista (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  direccion VARCHAR(100) NOT NULL,
  telefono VARCHAR(9) NOT NULL
);

CREATE TABLE hotel (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  direccion VARCHAR(100) NOT NULL,
  ciudad VARCHAR(25) NOT NULL,
  plazas INTEGER NOT NULL,
  telefono VARCHAR(9) NOT NULL
);

CREATE TABLE reserva (
  id_turista INT UNSIGNED NOT NULL,
  id_hotel INT UNSIGNED NOT NULL,
  fecha_entrada DATETIME NOT NULL,
  fecha_salida DATETIME NOT NULL,
  regimen ENUM('MP', 'PC'),
  PRIMARY KEY (id_turista,id_hotel),
  FOREIGN KEY (id_turista) REFERENCES turista(id),
  FOREIGN KEY (id_hotel) REFERENCES hotel(id)
);
```

Manipulación de Tablas

Crear una tabla - Opciones en la declaración de claves ajenas (FOREIGN KEY)

- **ON DELETE y ON UPDATE:** Nos permiten indicar el efecto que provoca el borrado o la actualización de los datos que están referenciados por claves ajenas. Las opciones que podemos especificar son las siguientes:
  - **RESTRICT**: Impide que se puedan actualizar o eliminar las filas que tienen valores referenciados por claves ajenas. Es la opción por defecto en MySQL.
  - **CASCADE:** Permite actualizar o eliminar las filas que tienen valores referenciados por claves ajenas.
  - **SET NULL:** Asigna el valor NULL a las filas que tienen valores referenciados por claves ajenas.
  - **NO ACTION:** Es una palabra clave del estándar SQL. En MySQL es equivalente a RESTRICT.
  - **SET DEFAULT:** No es posible utilizar esta opción cuando trabajamos con el motor de almacenamiento InnoDB. Puedes encontrar más información en la documentación oficial de MySQL.

Manipulación de Tablas

Crear una tabla - Opciones en la declaración de claves ajenas (FOREIGN KEY)

Ejemplo 1:

```
DROP DATABASE IF EXISTS proveedores;
CREATE DATABASE proveedores CHARSET utf8mb4;
USE proveedores;

CREATE TABLE categoria (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
);

CREATE TABLE pieza (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  color VARCHAR(50) NOT NULL,
  precio DECIMAL(7,2) NOT NULL,
  codigo_categoria INT UNSIGNED NOT NULL,
  FOREIGN KEY (codigo_categoria) REFERENCES categoria(codigo)
  ON DELETE RESTRICT
  ON UPDATE RESTRICT
);
```

```
INSERT INTO categoria VALUES (1, 'Categoria A');
INSERT INTO categoria VALUES (2, 'Categoria B');
INSERT INTO categoria VALUES (3, 'Categoria C');

INSERT INTO pieza VALUES (1, 'Pieza 1', 'Blanco', 25.90, 1);
INSERT INTO pieza VALUES (2, 'Pieza 2', 'Verde', 32.75, 1);
INSERT INTO pieza VALUES (3, 'Pieza 3', 'Rojo', 12.00, 2);
INSERT INTO pieza VALUES (4, 'Pieza 4', 'Azul', 24.50, 2);
```

- ¿Podríamos borrar la **Categoría A** de la tabla categoria?
- ¿Y la **Categoría C**?
- ¿Podríamos actualizar la **Categoría A** de la tabla categoria?

Manipulación de Tablas

Crear una tabla - Opciones en la declaración de claves ajenas (FOREIGN KEY)

Ejemplo 2:

```
DROP DATABASE IF EXISTS proveedores;
CREATE DATABASE proveedores CHARSET utf8mb4;
USE proveedores;

CREATE TABLE categoria (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
);

CREATE TABLE pieza (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  color VARCHAR(50) NOT NULL,
  precio DECIMAL(7,2) NOT NULL,
  codigo_categoria INT UNSIGNED NOT NULL,
FOREIGN KEY (codigo_categoria) REFERENCES
categoria(codigo)
  ON DELETE CASCADE ON
  UPDATE CASCADE
);
```

```
INSERT INTO categoria VALUES (1, 'Categoria A');
INSERT INTO categoria VALUES (2, 'Categoria B');
INSERT INTO categoria VALUES (3, 'Categoria C');

INSERT INTO pieza VALUES (1, 'Pieza 1', 'Blanco', 25.90, 1);
INSERT INTO pieza VALUES (2, 'Pieza 2', 'Verde', 32.75, 1);
INSERT INTO pieza VALUES (3, 'Pieza 3', 'Rojo', 12.00, 2);
INSERT INTO pieza VALUES (4, 'Pieza 4', 'Azul', 24.50, 2);
```

- ¿Podríamos borrar la **Categoría A** de la tabla categoria?
- ¿Qué le ocurre a las piezas que pertenecen la **Categoría A** después de borrarla?
- ¿Podríamos actualizar la **Categoría A** de la tabla categoria?
- ¿Qué le ocurre a las piezas que pertenecen la **Categoría A** después de actualizarla?

Manipulación de Tablas

Crear una tabla - Opciones en la declaración de claves ajenas (FOREIGN KEY)

Ejemplo 3:

```
DROP DATABASE IF EXISTS proveedores;
CREATE DATABASE proveedores CHARSET utf8mb4;
USE proveedores;

CREATE TABLE categoria (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
);

CREATE TABLE pieza (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  color VARCHAR(50) NOT NULL,
  precio DECIMAL(7,2) NOT NULL,
  codigo_categoria INT UNSIGNED,
  FOREIGN KEY (codigo_categoria) REFERENCES
categoria(codigo)
  ON DELETE SET NULL ON
  UPDATE SET NULL
);
```

```
INSERT INTO categoria VALUES (1, 'Categoria A');
INSERT INTO categoria VALUES (2, 'Categoria B');
INSERT INTO categoria VALUES (3, 'Categoria C');

INSERT INTO pieza VALUES (1, 'Pieza 1', 'Blanco', 25.90, 1);
INSERT INTO pieza VALUES (2, 'Pieza 2', 'Verde', 32.75, 1);
INSERT INTO pieza VALUES (3, 'Pieza 3', 'Rojo', 12.00, 2);
INSERT INTO pieza VALUES (4, 'Pieza 4', 'Azul', 24.50, 2);
```

- ¿Podríamos borrar la **Categoría A** de la tabla categoria?
- ¿Qué le ocurre a las piezas que pertenecen la **Categoría A** después de borrarla?
- ¿Podríamos actualizar la **Categoría A** de la tabla categoria?
- ¿Qué le ocurre a las piezas que pertenecen la **Categoría A** después de actualizarla

Lenguaje de Definición de Datos

- **FOREIGN KEY**

Nota

El estándar SQL define 5 opciones para manejar esta situación de tablas principal/secundaria de varias maneras. Estas opciones son:

- **ON DELETE CASCADE**: si se elimina una fila de la tabla principal, se eliminan todas las filas coincidentes en la tabla secundaria.
- **ON DELETE SET NULL**: si se elimina una fila de la tabla principal, todas las columnas de referencia en todas las filas coincidentes de la tabla secundaria se establecen en NULL.
- **ON DELETE SET DEFAULT**: si se elimina una fila de la tabla principal, todas las columnas de referencia en todas las filas coincidentes de la tabla secundaria se configuran en el valor predeterminado de la columna.
- **ON DELETE RESTRICT**: está prohibido eliminar una fila de la tabla principal si esa fila tiene alguna fila coincidente en la tabla secundaria. El punto en el tiempo cuando realiza la comprobación se puede aplazar hasta que se realice COMMIT.
- **ON DELETE NO ACTION** (el valor predeterminado): se prohíbe eliminar una fila de la tabla primaria si esa fila tiene filas coincidentes en la tabla secundaria.

Análoga a la opción ON DELETE hay una opción ON UPDATE. Define las mismas 5 opciones para el caso de cambiar una columna en la tabla principal a la que hace referencia la columna de una tabla secundaria.

- **ON UPDATE CASCADE**: Cualquier cambio en una columna referenciada en la tabla primaria provoca el mismo cambio en la columna de referencia correspondiente en las filas coincidentes de la tabla secundaria.
- **ON UPDATE SET NULL**: Cualquier cambio en una columna referenciada en la tabla primaria provoca que la columna de referencia correspondiente en las filas coincidentes de la tabla secundaria se establezca como nula.
- **ON UPDATE SET DEFAULT**: Cualquier cambio en una columna referenciada en la tabla principal provoca que la columna de referencia correspondiente en las filas coincidentes de la tabla de secundaria se establezca en su valor predeterminado.
- **ON UPDATE RESTRICT**: está prohibido cambiar una fila de la tabla principal si esa fila tiene filas coincidentes en la tabla secundaria. El punto en el tiempo cuando se realiza la comprobación se puede aplazar hasta que se realice COMMIT.
- **ON UPDATE NO ACTION** (valor predeterminado): está prohibido cambiar una fila de la tabla principal si esa fila tiene alguna fila coincidente en la tabla secundaria.

Si ON DELETE o ON UPDATE no están especificados, se producirá la acción predeterminada NO ACTION. En algunos sistemas, NO ACTION se implementa en el sentido de la opción RESTRICT.

Nombrando las restricciones

Aunque hemos visto que podemos añadir las restricciones asociadas a cada campo, es mucho mejor definirlas tras todos los campos, haciendo uso de la keyword **CONSTRAINT** poniéndole un nombre a la restricción. Así pues, como norma general, nombraremos las restricciones con ***pk, fk, uk,*** , etc... y utilizando la barra de subrayado como separador, seguido del nombre de la tabla y/o los campos implicados.
Al asignarles un nombre, más adelante podremos acceder a las restricciones para modificarlas o eliminarlas.

De este modo, la tabla anterior queda mucho mejor si la definimos mediante:

```
CREATE TABLE categoria (
 codigo INT UNSIGNED AUTO_INCREMENT,
 nombre VARCHAR(100) NOT NULL
CONSTRAINT pk_codigo PRIMARY KEY (codigo),
);
```

Claves ajenas

Para indicar las claves ajenas es mucho mejor acostumbrarse a definirlas siempre como restricciones. Para ello, usaremos la siguiente estructura, donde ***col*** es el campo que apunta a la clave primaria clave de la tabla tabla: *CONSTRAINT nombre FOREIGN KEY col REFERENCES tabla (clave)*

Cuando tenemos una clave primaria o clave ajena compuesta, sólo hemos de indicar los atributos separándolos con comas:

```
CREATE TABLE EXISTIR (
   tipo CHAR(9),
   modelo INT,
   numAlmacen INT,
   cantidad DECIMAL(7),
CONSTRAINT pk_existir PRIMARY KEY(tipo, modelo, numAlmacen),
CONSTRAINT fk_exi_pie FOREIGN KEY (tipo, modelo) REFERENCES PIEZA,
CONSTRAINT fk_exi_alm FOREIGN KEY (numAlmacen) REFERENCES ALMACEN
   );
```

Lenguaje de Definición de Datos

Tipos de datos

Números enteros

Tipo	Bytes	Mínimo	Máximo
TINYINT	1	-128	127
TINYINT UNSIGNED	1	0	255
SMALLINT	2	-32768	32767
SMALLINT UNSIGNED	2	0	65535
MEDIUMINT	3	-8388608	8388607
MEDIUMINT UNSIGNED	3	0	16777215
INT	4	-2147483648	2147483647
INT UNSIGNED	4	0	4294967295
INTEGER	4	-2147483648	2147483647
INTEGER UNSIGNED	4	0	4294967295
BIGINT	8	-9223372036854775808	9223372036854775807
BIGINT UNSIGNED	8	0	18446744073709551615

Tipos de datos

Números enteros

ZEROFILL

Todos los tipos de datos numéricos admite el atributo ZEROFILL. Cuando asignamos este atributo a una columna también se le añade de forma automática el atributo UNSIGNED, de modo que el campo quedaría como UNISGNED ZEROFILL.

INT(11) no quiere decir que queremos guardar un número entero de 11 cifras. El número indicado entre paréntesis indica el ancho de la columna que ocupará dicho valor y tiene utilidad cuando asignamos el atributo UNSIGNED ZEROFILL. En este caso se completa con 0 a la izquierda del valor hasta alcanzar el número indicado entre paréntesis.

Por ejemplo, para una columna declarada como **INT(4) ZEROFILL**, el valor 5 será representado como **0005.**

Lenguaje de Definición de Datos

Tipos de datos

BIT, BOOL, BOOLEAN, SERIAL

Tipo	Descripción
`BIT(M)`	`M` puede ser un valor de 1 a 64.
	Indica el número de bits que vamos a utilizar para este campo.
	Si se omite el valor de `M` se utiliza 1 bit por defecto.
`BOOL, BOOLEAN`	Son equivalentes a `TINYINT(1)`.
	El valor 0 se considera como `FALSE`.
	Cualquier valor distinto de 0 será `TRUE`.
`SERIAL`	Es un alias para: `BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE`.

Lenguaje de Definición de Datos

Tipos de datos

Números de punto flotante

Los tipos de datos FLOAT y DOUBLE almacenan valores numéricos aproximados y no valores exactos, por lo tanto, debe tener en cuenta que podemos obtener resultados erróneos a la hora de realizar comparaciones exactas.

Tipo	Bytes	Mínimo	Máximo
FLOAT	4		
FLOAT(M,D)	4	±1.175494351E-38	±3.402823466E+38
FLOAT(M,D)UNSIGNED	4	1.175494351E-38	3.402823466E+38
DOUBLE	8		
DOUBLE(M,D)	8	±1.7976931348623157E+308	±2.2250738585072014E-308
DOUBLE(M,D)UNSIGNED	8	1.7976931348623157E+308	2.2250738585072014E-308

- M: indica el número de dígitos en total (la precisión).
- D : es el número de cifras decimales.

Lenguaje de Definición de Datos

MySQL permite utilizar una sintaxis no estándar para definir los tipos de datos FLOAT y DOUBLE como: FLOAT (M,D) y DOUBLE(M,D). Donde (M,D) representan que los valores pueden ser almacenados con M dígitos en total (parte entera más parte decimal), de los cuales D dígitos serán para la parte decimal. A partir de la versión 8.0.17 de MySQL esta sintaxis está obsoleta.

Por ejemplo, un número declarado como FLOAT(7,4) tendrá 7 dígitos como máximo y 4 de ellos serán decimales. El rango de números que se pueden representar en este caso será desde -999.9999 hasta 999.9999.

El estándar SQL permite indicar de forma opcional el número de bits (precisión) que se van a utilizar para almacenar la mantisa en los datos de tipo FLOAT. En este caso, el número de bits (precisión) se indicará entre paréntesis a continuación de la palabra reservada FLOAT, por ejemplo: FLOAT(p), donde p indica el número de bits de la mantisa.

El tipo de dato FLOAT representa un número real de 32 bits en simple precisión, con 1 bit para el signo, 8 bits para el exponente y 23 para la mantisa, por lo tanto, a la hora de definir una precisión para este tipo de dato podremos utilizar un valor entre 0 y 23 bits.

Puedes encontrar más información sobre números en punto flotante en la documentación oficial de MySQL.

Tipos de datos

Números en punto fijo (Valores exactos)

Los tipos de datos DECIMAL y NUMERIC almacenan valores numéricos exactos y se utilizan cuando es necesario guardar los valores exactos sin redondeos. Se suelen utilizar cuando trabajamos con datos monetarios.

Tipo	Bytes
DECIMAL	
DECIMAL(M,D)	M + 2 bytes si D > 0
DECIMAL(M,D) UNSIGNED	M + 1 bytes si D = 0
NUMERIC	
NUMERIC(M,D)	M + 2 bytes si D > 0
NUMERIC(M,D) UNSIGNED	M + 1 bytes si D = 0

En MySQL los tipos de datos DECIMAL y NUMERIC son equivalentes.

- M indica el número de dígitos en total (la precisión). Tiene un rango de 1 a 65.
- D es el número de cifras decimales. Tiene un rango de 0 a 30.

Por ejemplo, un número declarado como DECIMAL(7,4) tendrá 7 dígitos como máximo y 4 de ellos serán decimales. El rango de números que se pueden representar en este caso será desde -999.9999 hasta 999.9999. Cuando se declara una columna como DECIMAL y no se indica la precisión (M) ni el número de cifras decimales (D), se utilizarán los valores por defecto, que es equivalente a declarar la columna como DECIMAL(10,0). Si sólo se indica la precisión (M) y no se indica el número de cifras decimales (D), entonces la columna no almacenará decimales. Por ejemplo, si declaramos una columna como DECIMAL(7), es equivalente a declararla como DECIMAL(7,0).

Lenguaje de Definición de Datos

Tipos de datos

Fechas y tiempo

Tipo	Bytes	Descripción	Rango	Máximo
DATE	3	YYYY-MM-DD	1000-01-01	9999-12-31
DATETIME	8	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00	9999-12-31 23:59:59
TIMESTAMP	4	YYYY-MM-DD HH:MM:SS	1970-01-01 00:00:00	2038-01-19 03:14:07
TIME	3	HH:MM:SS	-838:59:59	838:59:59
YEAR[(2|4)]	1	YY o YYYY	YY: 70 (1970)	YY: 69 (2069)
			YYYY: 1901	YYYY: 2155

Lenguaje de Definición de Datos

Tipos de datos

Cadenas de caracteres

Tipo	Descripción
CHAR(M)	0 <= M <= 255
VARCHAR(M)	0 <= M <= 65535
TEXT[(M)]	L + 2 bytes, donde L < 2^{16} = 65536
MEDIUMTEXT	L + 3 bytes, donde L < 2^{24} = 16 MB
LONGTEXT	L + 4 bytes, donde L < 2^{32} = 4 GB

Datos binarios

Tipo	Descripción
BINARY	0 <= M <= 255
VARBINARY	0 <= M <= 65535
BLOB	L + 2 bytes, donde L < 2^{16} = 65536
MEDIUMBLOB	L + 3 bytes, donde L < 2^{24} = 16 MB
LONGBLOB	L + 4 bytes, donde L < 2^{32} = 4 GB

ENUM y SET

Tipo	Descripción
ENUM('valor1', 'valor2', ...)	Puede tener 65535 valores. Sólo permite seleccionar un valor de la lista
SET('valor1', 'valor2', ...)	Puede tener 64 valores. Permite seleccionar varios valores de la lista

Lenguaje de Definición de Datos

Tipos de datos

Resumen de tipos de datos disponibles en MySQL

```
BIT[(length)]
TINYINT[(length)] [UNSIGNED] [ZEROFILL]
SMALLINT[(length)] [UNSIGNED] [ZEROFILL]
MEDIUMINT[(length)] [UNSIGNED] [ZEROFILL]
INT[(length)] [UNSIGNED] [ZEROFILL]
INTEGER[(length)] [UNSIGNED] [ZEROFILL]
BIGINT[(length)] [UNSIGNED] [ZEROFILL]
REAL[(length,decimals)] [UNSIGNED] [ZEROFILL]
DOUBLE[(length,decimals)] [UNSIGNED] [ZEROFILL]
FLOAT[(length,decimals)] [UNSIGNED] [ZEROFILL]
DECIMAL[(length[,decimals])] [UNSIGNED] [ZEROFILL]
NUMERIC[(length[,decimals])] [UNSIGNED] [ZEROFILL]
DATE
TIME
TIMESTAMP
DATETIME
YEAR
```

```
CHAR[(length)] [BINARY] [CHARACTER SET charset_name] [COLLATE collation_name]
VARCHAR(length) [BINARY] [CHARACTER SET charset_name] [COLLATE collation_name]
BINARY[(length)]
VARBINARY(length)
TINYBLOB
BLOB
MEDIUMBLOB
LONGBLOB
TINYTEXT [BINARY] [CHARACTER SET charset_name] [COLLATE collation_name]
TEXT [BINARY] [CHARACTER SET charset_name] [COLLATE collation_name]
MEDIUMTEXT [BINARY] [CHARACTER SET charset_name] [COLLATE collation_name]
LONGTEXT [BINARY] [CHARACTER SET charset_name] [COLLATE collation_name]
ENUM(value1,value2,value3,...) [CHARACTER SET charset_name] [COLLATE
collation_name]
SET(value1,value2,value3,...) [CHARACTER SET charset_name]
[COLLATE collation_name]
spatial_type
```

Tipos de datos

Valores fuera de rango y desbordamientos (*Out-of-Range and Overflow*)

Cuando MySQL almacena en una columna de tipo numérico un valor que está fuera del rango permitido, pueden ocurrir dos situaciones que dependen de la configuración de MySQL (sql_mode).

- Si está habilitado el modo estricto, MySQL no permite que se inserten los valores que están fuera de rango y lanza un mensaje de error.

  Ejemplo. Si tenemos una columna con un tipo de dato TINYINT UNSIGNED, el rango de valores permitido para esta columna será [0, 255]. Si quisiéramos almacenar el valor 256 en esta columna MySQL lanzaría un mensaje de error con el código 1264.

```
SET sql_mode = 'TRADITIONAL';
CREATE TABLE test (data TINYINT UNSIGNED);
INSERT INTO test VALUES(256);
ERROR 1264 (22003): Out of range value for column 'data' at row 1
```

Si consultamos el contenido de la tabla veremos que no se ha insertado ningún valor.

```
SELECT * FROM test;
Empty set (0.00 sec)
```

Tipos de datos

Valores fuera de rango y desbordamientos (*Out-of-Range and Overflow*)

- Si no está habilitado el modo estricto, MySQL permite se que inserten los valores que están fuera de rango pero se adaptan al rango de valores que permita la columna.

  Ejemplo. Si tenemos una columna con un tipo de dato TINYINT UNSIGNED, el rango de valores permitido para esta columna será [0, 255]. Si quisiéramos almacenar el valor 400 en esta columna y MySQL está configurado en modo no estricto, se almacenaría 255 que es el máximo valor que se puede representar con este tipo de dato.

```
SET sql_mode = '';
CREATE TABLE test (data TINYINT UNSIGNED);
INSERT INTO test VALUES(400);
1 row(s) affected, 1 warning(s): 1264 Out of range value for column 'data' at row 1
```

Si consultamos el contenido de la tabla veremos que en lugar de almacenar el valor `400` se ha almacenado el valor `255`.

```
SELECT * FROM test;
+------+
| data |
+------+
| 255 |
+------+
```

Tipos de datos

Valores fuera de rango y desbordamientos (*Out-of-Range and Overflow*)

Cómo configurar la variable sql_mode

Esta variable se puede configurar a nivel global o a nivel de sesión.

```
SET GLOBAL sql_mode = 'modes';
SET SESSION sql_mode = 'modes';
```

Donde algunos de los valores más importantes que podemos utilizar en modes son:

- ANSI
- STRICT_TRANS_TABLES
- TRADITIONAL

Cómo consultar la variable sql_mode

Para consultar cuál es el valor que tienen estas variables podemos hacerlo así:

```
SELECT @@GLOBAL.sql_mode;
SELECT @@SESSION.sql_mode;
```

LENGUAJE DE DEFINICIÓN DE DATOS (DDL) (OPERADORES)

OPERADORES ARITMÉTICOS

Operan entre valores numéricos y devuelven un valor numérico como resultado de realizar los cálculos indicados, algunos de ellos se pueden utilizar también con fechas. Se emplean para realizar cálculos numéricos.

Los operadores aritméticos son:

+	Suma
-	Resta
*	Multiplicación
/	División
Div	División entera (parte entera de la división, sin decimales)

LENGUAJE DE DEFINICIÓN DE DATOS (DDL) (OPERADORES)

OPERADORES COMPARACIÓN

Las expresiones formadas con operadores de comparación dan como resultado un valor de tip Verdadero/Falso/Nulo (True/False/Null).

Los operadores de comparación son:

=	Igual
!=	Distinto
<>	Distinto
<	Menor
<=	Menor o igual
>	Mayor
>=	Mayor o igual

BETWEEN / NOT BETWEEN
IN / NOT IN
IS NULL / IS NOT NULL
LIKE

Ejemplos:

```
horas >= 10.5
nombre = 'PEPE'
fecha < '1-ene-93'
```

```
nombre LIKE 'Pedro%'
codigo NOT LIKE 'cod1_'
```

```
horas BETWEEN 10 AND 20    -- que equivale a horas >= 10 AND horas <= 20
```

Lenguaje de Definición de Datos

OPERADORES COMPARACIÓN. Los primeros son los operadores relaciones ya conocidos. Los segundos son pares de un operador y su negación con la siguiente forma de actuar:

- **BETWEEN valor1 AND valor2 .** Da como resultado VERDADERO si el valor comparado es mayor o igual que valor1 y menor o igual que valor2 y FALSO en el caso contrario.
- **IN (lista de valores separados por comas).** Da como resultado VERADADERO si el valor comparado está dentro de la lista de valores especificado y FALSO en el caso contrario
- **IS NULL.** Da como resultado VERDADERO si el valor del dato comparado es nulo (NULL) y FALSO en el caso contrario
- **LIKE.** Permite comparar dos cadenas de caracteres con la peculiaridad de que admite caracteres comodines. Los caracteres comodines son '%' y '_'. Estos caracteres permiten utilizar patrones en la comparación. El '%'puede ser sustituido por un grupo de caracteres (de 0 a cualquier número) y el '_' por un carácter cualquiera en esa posición.

OPERADORES LOGICOS

Operan entre datos con valores lógicos Verdadero/Falso/Nulo y devuelven el valor lógico Verdadero/Falso/Nulo.

Los operadores lógicos son:

!	NOT
&&	AND
||	OR
	XOR

Podemos establecer:

- El **operador NOT** devuelve VERDADERO cuando el operando es falso, y FALSO cuando el operando es verdadero y NULO cuando el operando es nulo.
- El **operador AND** devolverá VERDADERO cuando los dos operandos sean verdaderos, FALSO cuando alguno de los dos operandos sea falso y NULO en los demás casos.
- El **operador OR** devolverá VERDADERO cuando alguno de los operandos sea verdadero, FALSO cuando los dos operandos sean falsos;y NULO en los demás casos.
- El **operador XOR** devolverá VERDADERO si uno de los operandos es verdadero y el otro falso, FALSO cuando ambos sean verdaderos o ambos falsos y NULO si alguno de ellos es nulo.

Lenguaje de Definición de Datos

NOT	VERDADERO	FALSO	NULO
	FALSO	VERDADERO	NULO

AND	VERDADERO	FALSO	NULO
VERDADERO	VERDADERO	FALSO	NULO
FALSO	FALSO	FALSO	FALSO
NULO	NULO	FALSO	NULO

OR	VERDADERO	FALSO	NULO
VERDADERO	VERDADERO	VERDADERO	VERDADERO
FALSO	VERDADERO	FALSO	NULO
NULO	VERDADERO	NULO	NULO

XOR	VERDADERO	FALSO	NULO
VERDADERO	FALSO	VERDADERO	NULO
FALSO	VERDADERO	FALSO	NULO
NULO	NULO	NULO	NULO

Ejemplos:

```
nombre = 'PEPE' OR horas BETWEEN 10 AND 20
horas > 10 AND telefono IS NULL
NOT (nombre IN ('PEPE','LUIS'))
```

LENGUAJE DE DEFINICIÓN DE DATOS (DDL) (FUNCIONES)

FUNCIONES PREDEFINIDAS

Son funciones incorporadas por el gestor y son muy utilizadas en SQLy dan mucha potencia al lenguaje. Cabe subrayar que las funciones no modifican los valores del argumento, indicado entre paréntesis, sino que devuelven un valor creado a partir de los argumentos que se le pasan en la llamada, y ese valor puede ser utilizado en cualquier parte de una sentencia SQL.

Existen muchas funciones predefinidas para operar con todo tipo de datos. A continuación se indican las funciones predefinidas más utilizadas. Estas funciones se tratarán con más detalle en la siguiente unidad de trabajo.

Lenguaje de Definición de Datos

Funciones numéricas o aritméticas

Función	Valor que devuelve.
ABS(*num*)	**Valor absoluto.** Valor absoluto de *num.*
CEIL(*num)*	**Función "Techo"** Devuelve el entero mas pequeño mayor que *num*
FLOOR(*num*)	**Función "Suelo"** Devuelve el entero mas grande menor que *num*
EXP(*num*)	**Potencia del número e** Devuelve el número e elevado a *num*
LN(*num*)	**Logaritmo neperiano** Devuelve el logaritmo en base e de *num*
LOG(*num*)	**Logaritmo** Devuelve el logaritmo en base 10 de *num*
MOD(*num1, num2).*	**Módulo** Resto de la división entera de *num1* por *num2*
PI()	**Pi** Devuelve el valor de la constante PI
POWER(*num1, num2).*	**Potencia** Devuelve *num1* elevado a *num2.*
RAND*()*	**Número aleatorio** Genera un número aleatorio entre 0 y 1
ROUND(*num1, num2)*	**Redondeo** Devuelve *num1* redondeado a *num2* decimales. Si se omite *num2* redondea a 0 decimales.
SIGN(*num*).	**Signo** Si *num* < 0 devuelve -1, Si *num* = 0 devuelve 0 Si *num* > 0 devuelve 1.
SQRT(*num*).	**Raíz cuadrada** Devuelve la raíz cuadrada de *num*
TRUNCATE(*num1, num2).*	**Truncado** Devuelve *num1* truncado a *num2* decimales. Si se omite *num2* trunca a 0 decimales.

Lenguaje de Definición de Datos

Funciones de caracteres

Función	Valor que devuelve.
ASCII(*cad1*).	**ASCII** Código ASCII del carácter *cad1*.
CHAR(*num*)	**Carácter ASCII** Devuelve el carácter cuyo código ASCII es *num*
CONCAT(*cad1,cad2 [,cad3…]*.)	**Concatenar** Concatena *cad1* con *cad2. Si* existiesen más cadenas, *cad3…*, las concatenaría a continuación
INSERT(*cad1, pos, ,len, cad2)*	**Insertar** Devuelve *cad1* con *len* caracteres desde *pos* en adelante sustituidos en *cad2*
LENGTH(*cad1*)	**Longitud** Devuelve la longitud de *cad1*.
LOCATE(*cad1,cad2,pos)*	**Localizar** Devuelve la posición de la primera ocurrencia de *cad1* en *cad2* empezando desde *pos*
LOWER(*cad1*)	**Minúsculas** La cadena *cad1* en minúsculas.

LPAD(*cad1, n, cad2)*	**Rellenar (Izquierda)** Añade a *cad1* por la izquierda *cad2*, hasta que tenga n caracteres. Si se omite *cad2,* añade blancos.
LTRIM(*cad1*)	**Suprimir (Izquierda)** Suprime blancos a la izquierda de *cad1.*
REPLACE(*cad1,cad2,cad3)*	**Reemplazar** Devuelve *cad1* con todas las ocurrencias de *cad2* reemplazadas por *cad3*
RPAD(*cad1, n, cad2)*	**Rellenar (Derecha)** Igual que LPAD pero por la derecha.
RTRIM(*c1*)	**Suprimir (Derecha)** Suprime blancos a la derecha de *c1.* Igual que LTRIM pero por la izquierda.
SUBSTR(*c1, n, m)*	**Subcadena** Devuelve una subcadena a partir de *c1* comenzando en La posición *n* tomando *m* caracteres.
UPPER(*cad1*)	**Mayúsculas** La cadena *cad1* en mayúsculas.

Lenguaje de Definición de Datos

· Funciones de fecha

Función	Valor que devuelve.
ADDDATE(*Fecha, Num*)	**Incremento de días** Devuelve *Fecha* incrementada en *Num* días
SUBDATE(*Fecha, Num*)	**Decremento de días** Devuelve *Fecha* decrementada en *Num* días
DATE_ADD(*Fecha*, INTERVAL Num *Formato*)	**Incremento** Devuelve *Fecha* incrementada en Num veces lo indicado en *Formato* *Formato* puede ser entre otros: DAY, WEEK, MONTH, YEAR, HOUR, MINUTE, SECOND
DATE_SUB(Fecha, INTERVAL num *Formato*)	**Decremento** Devuelve Fecha decrementada en *Num* veces lo indicado en *Formato* *Formato* puede ser entre otros: DAY, WEEK, MONTH, YEAR, HOUR, MINUTE, SECOND
DATEDIFF(*Fecha1,Fecha2*)	**Diferencia de fechas** Devuelve el número de días entre *Fecha1* y *Fecha2*
DAYNAME(*Fecha*)	**Nombre del día de la semana** Devuelve el nombre del día de la semana de *Fecha*
DAYOFMONTH(*Fecha*)	**Día del mes** Devuelve el número del día del mes de *Fecha*
DAYOFWEEK(*Fecha*)	**Día de la semana** Devuelve el número del día de la semana de *Fecha* (1.Domingo, 2:Lunes.....7:Sábado)
DAYOFYEAR(*Fecha*)	**Día del año** Devuelve el número de día del año de *Fecha* (de 1 a 366)
WEEKOFYEAR(*Fecha*)	**Semana** Devuelve el número de semana de *Fecha* (de 1 a 53)
MONTH(*Fecha*)	**Mes** Devuelve el número de mes de *Fecha* (de 1 a 12)
YEAR(*Fecha)*	**Año** Devuelve el número de año con 4 dígitos de *Fecha* (de 0000 a 9999)
HOUR(*Tiempo)*	**Hora** Devuelve la hora de *Tiempo* (de 0 a 23)
MINUTE(*Tiempo*)	**Minutos** Devuelve los minutos de *Tiempo* (de 0 a 59)
SECOND(*Tiempo*)	**Segundos** Devuelve los segundos de *Tiempo* (de 0 a 59)
CURDATE()	Devuelve la fecha actual con el formato 'YYYY-MM-DD'
CURTIME()	Devuelve la hora actual con el formato 'HH:MM:SS
SYSDATE()	Devuelve la fecha y la hora actual con el formato 'YYYY-MM-DD HH:MM:SS'

Funciones de comparación

Función	Valor que devuelve.
GREATEST(*lista de valores)*	**Mayor de la lista** Devuelve el valor más grande de una lista de columnas o expresiones de columna
LEAST(*lista de valores)*	**Menor de la lista** Devuelve el valor más pequeño de una lista de columnas o expresiones de cdumna
IFNULL *(exp1, exp2)*	**Conversión de nulos** Si *exp1* es nulo devuelve *exp2,* sino devuelve *exp1*
ISNULL(*exp)*	**Comprobación de nulo** Devuelve 1(`True`) si *exp* es `NULL` y 0 (`False`) en caso contrario
STRCMP(*cad1,cad2*)	**Comparación de cadenas** Devuelve 1(`True`) si *cad1* y *cad2* son iguales, 0(`False`) si no lo son y `NULL` si alguna de ellas es nula

Crear una tabla - Opciones a tener en cuenta en la creación de las tablas

Algunas de las opciones que podemos indicar durante la creación de las tablas son las siguientes:

- AUTO_INCREMENT: Aquí podemos indicar el valor inicial que vamos a usar en el campo definido como AUTO_INCREMENT.
- CHARACTER SET: Especifica el set de caracteres que vamos a utilizar en la tabla.
- COLLATE: Especifica el tipo de cotejamiento que vamos a utilizar en la tabla.
- ENGINE: Especifica el motor de almacenamiento que vamos a utilizar para la tabla. Los más habituales en MySQL son InnoDB y MyISAM. Por defecto las tablas se crean con el motor InnoDB

Para conocer todas las opciones posibles podemos consultar la sintaxis de creación de tablas en la documentación oficial de MySQL. Con el objetivo de simplificar la creación de tablas solamente hemos enumerado las opciones con las que vamos a trabajar durante el curso.

En la documentación oficial de MySQL podemos encontrar más información sobre los diferentes motores de almacenamiento disponibles en MySQL.

Lenguaje de Definición de Datos

Manipulación de Tablas

Crear una tabla - Opciones a tener en cuenta en la creación de las tablas

Ejemplo:

```
DROP DATABASE IF EXISTS proveedores;
CREATE DATABASE proveedores CHARSET utf8mb4;
USE proveedores;

CREATE TABLE categoria (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 AUTO_INCREMENT=1000;

CREATE TABLE pieza (
  codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  color VARCHAR(50) NOT NULL,
  precio FLOAT NOT NULL,
  codigo_categoria INT UNSIGNED NOT NULL,
  FOREIGN KEY (codigo_categoria) REFERENCES categoria(codigo)
) ENGINE=InnoDB DEFAULT CHARSET=utf8 AUTO_INCREMENT=1000;
```

En este ejemplo se ha seleccionado para cada una de las tablas las siguientes opciones de configuración: InnoDB como motor de base de datos, utf8 como el set de caracteres y el valor 1000 como valor inicial para las columnas de tipo AUTO_INCREMENT.

Manipulación de Tablas

Eliminar una tabla

```
DROP [TEMPORARY] TABLE [IF EXISTS] nombre_tabla [, nombre_tabla];
```

Ejemplos:

```
DROP TABLE nombre_tabla;

DROP TABLE IF EXISTS nombre_tabla;

DROP TABLE nombre_tabla_1, nombre_tabla_2;
```

Lenguaje de Definición de Datos

Manipulación de Tablas

Modificar una tabla: En muchas ocasiones es necesario modificar los atributos de una tabla, añadir nuevos campos o eliminar otros. Si la tabla no tiene datos podemos eliminar la tabla y volver a crearla, pero si se trata de una tabla que ya contiene datos tenemos que hacer uso de la sentencia ALTER TABLE.

```
ALTER TABLE tbl_name
    [alter_specification [, alter_specification] ...]
    [partition_options]

alter_specification:
    table_options
  | ADD [COLUMN] col_name column_definition
        [FIRST | AFTER col_name]
  | ADD [COLUMN] (col_name column_definition,...)
  | ADD {INDEX|KEY} [index_name]
        [index_type] (index_col_name,...) [index_option] ...
  | ADD [CONSTRAINT [symbol]] PRIMARY KEY
        [index_type] (index_col_name,...) [index_option] ...
  | ADD [CONSTRAINT [symbol]]
        UNIQUE [INDEX|KEY] [index_name]
        [index_type] (index_col_name,...) [index_option] ...
  | ADD FULLTEXT [INDEX|KEY] [index_name]
        (index_col_name,...) [index_option] ...
```

```
  | ADD SPATIAL [INDEX|KEY] [index_name]
        (index_col_name,...) [index_option] ...
  | ADD [CONSTRAINT [symbol]]
        FOREIGN KEY [index_name] (index_col_name,...)
        reference_definition
  | ALGORITHM [=] {DEFAULT|INPLACE|COPY}
  | ALTER [COLUMN] col_name {SET DEFAULT literal | DROP
DEFAULT}
  | CHANGE [COLUMN] old_col_name new_col_name
column_definition
        [FIRST|AFTER col_name]
  | [DEFAULT] CHARACTER SET [=] charset_name [COLLATE
[=] collation_name]
  | CONVERT TO CHARACTER SET charset_name [COLLATE
collation_name]
  | {DISABLE|ENABLE} KEYS
  | {DISCARD|IMPORT} TABLESPACE
```

Lenguaje de Definición de Datos

Manipulación de Tablas

Modificar una tabla

```
| DROP [COLUMN] col_name
| DROP {INDEX|KEY} index_name
| DROP PRIMARY KEY
| DROP FOREIGN KEY fk_symbol
| FORCE
| LOCK [=] {DEFAULT|NONE|SHARED|EXCLUSIVE}
| MODIFY [COLUMN] col_name column_definition
    [FIRST | AFTER col_name]
| ORDER BY col_name [, col_name] ...
| RENAME {INDEX|KEY} old_index_name TO new_index_name
| RENAME [TO|AS] new_tbl_name
| {WITHOUT|WITH} VALIDATION
| ADD PARTITION (partition_definition)
| DROP PARTITION partition_names
| DISCARD PARTITION {partition_names | ALL} TABLESPACE
| IMPORT PARTITION {partition_names | ALL} TABLESPACE
```

```
| TRUNCATE PARTITION {partition_names | ALL}
| COALESCE PARTITION number
| REORGANIZE PARTITION partition_names INTO
(partition_definitions)
| EXCHANGE PARTITION partition_name WITH TABLE
tbl_name [{WITH|WITHOUT} VALIDATION]
| ANALYZE PARTITION {partition_names | ALL}
| CHECK PARTITION {partition_names | ALL}
| OPTIMIZE PARTITION {partition_names | ALL}
| REBUILD PARTITION {partition_names | ALL}
| REPAIR PARTITION {partition_names | ALL}
| REMOVE PARTITIONING
    | UPGRADE PARTITIONING
```

Lenguaje de Definición de Datos

Manipulación de Tablas

Modificar una tabla

```
index_col_name:
    col_name [(length)] [ASC | DESC]

index_type:
    USING {BTREE | HASH}

index_option:
    KEY_BLOCK_SIZE [=] value
  | index_type
  | WITH PARSER parser_name
  | COMMENT 'string'

table_options:
    table_option [[,] table_option] ...
```

```
table_option:
    AUTO_INCREMENT [=] value
  | AVG_ROW_LENGTH [=] value
  | [DEFAULT] CHARACTER SET [=] charset_name
  | CHECKSUM [=] {0 | 1}
  | [DEFAULT] COLLATE [=] collation_name
  | COMMENT [=] 'string'
  | COMPRESSION [=] {'ZLIB'|'LZ4'|'NONE'}
  | CONNECTION [=] 'connect_string'
  | {DATA|INDEX} DIRECTORY [=] 'absolute path to directory'
  | DELAY_KEY_WRITE [=] {0 | 1}
  | ENCRYPTION [=] {'Y' | 'N'}
  | ENGINE [=] engine_name
  | INSERT_METHOD [=] { NO | FIRST | LAST }
  | KEY_BLOCK_SIZE [=] value
  | MAX_ROWS [=] value
  | MIN_ROWS [=] value
  | PACK_KEYS [=] {0 | 1 | DEFAULT}
  | PASSWORD [=] 'string'
  | ROW_FORMAT [=]
{DEFAULT|DYNAMIC|FIXED|COMPRESSED|REDUNDANT|COMPACT}
  | STATS_AUTO_RECALC [=] {DEFAULT|0|1}
  | STATS_PERSISTENT [=] {DEFAULT|0|1}
  | STATS_SAMPLE_PAGES [=] value
  | TABLESPACE tablespace_name [STORAGE
  {DISK|MEMORY|DEFAULT}]
  | UNION [=] (tbl_name[,tbl_name]...)

partition_options:
        (see CREATE TABLE options)
```

Manipulación de Tablas

Ejemplo de ALTER TABLE <tbl_name> MODIFY

MODIFY nos permite modificar el tipo de dato de una columna y sus atributos. Suponemos que tenemos la siguiente tabla creada

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(25)
);
```

Y queremos modificar la columna nombre para que pueda almacenar 50 caracteres y además que sea NOT NULL. En este caso usaremos la sentencia:

```
ALTER TABLE usuario MODIFY nombre VARCHAR(50) NOT NULL;
```

Después de ejecutar esta sentencia la tabla quedaría así:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL
);
```

Manipulación de Tablas

Ejemplo de ALTER TABLE <tbl_name> CHANGE

CHANGE nos permite renombrar una columna, modificar el tipo de dato de una columna y sus atributos. Suponemos que tenemos la siguiente tabla creada

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre_de_usuario VARCHAR(25)
);
```

Y queremos renombrar el nombre de la columna nombre_de_usuario como nombre, que pueda almacenar 50 caracteres y además que sea NOT NULL. En este caso usaremos la sentencia:

```
ALTER TABLE usuario CHANGE nombre_de_usuario nombre VARCHAR(50) NOT NULL;
```

Después de ejecutar esta sentencia la tabla quedaría así:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL
);
```

Manipulación de Tablas

Ejemplo de ALTER TABLE <tbl_name> ALTER

ALTER nos permite asignar un valor por defecto a una columna o eliminar el valor por defecto que tenga establecido. Suponemos que tenemos la siguiente tabla creada

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  sexo ENUM('H', 'M') NOT NULL
);
```

Y queremos que el valor por defecto de la columna sexo sea M. En este caso usaremos la sentencia:

```
ALTER TABLE usuario ALTER sexo SET DEFAULT 'M';
```

Después de ejecutar esta sentencia la tabla quedaría así:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(50) NOT NULL,
  sexo ENUM('H', 'M') NOT NULL DEFAULT 'M'
);
```

Si ahora quisiéramos eliminar el valor por defecto de la columna sexo, usaríamos la siguiente sentencia:

```
ALTER TABLE usuario ALTER sexo DROP DEFAULT;
```

Manipulación de Tablas

Ejemplo de ALTER TABLE <tbl_name> ADD

ADD nos permite añadir nuevas columnas a una tabla. Con los modificadores FIRST y AFTER podemos elegir el lugar de la tabla donde queremos insertar la nueva columna. FIRST coloca la nueva columna en primer lugar y AFTER la colocaría detrás de la columna que se especifique. Si no se especifica nada la nueva columna se añadiría detrás de la última columna de la tabla. Suponemos que tenemos la siguiente tabla creada

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(50) NOT NULL,
  sexo ENUM('H', 'M') NOT NULL
);
```

Y queremos añadir la columna fecha_nacimiento de tipo DATE:

```
ALTER TABLE usuario ADD fecha_nacimiento DATE NULL;
```

Manipulación de Tablas

Ejemplo de ALTER TABLE <tbl_name> ADD

En este caso la nueva columna se ha añadido detrás de la última columna, sexo. La tabla quedaría así:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  sexo ENUM('H', 'M') NOT NULL,
  fecha_nacimiento DATE NOT NULL
);
```

Suponemos que ahora queremos añadir las columnas `apellido1` y `apellido2` detrás de la columna `nombre`.

```
ALTER TABLE usuario ADD apellido1 VARCHAR(50) NOT NULL AFTER nombre;
ALTER TABLE usuario ADD apellido2 VARCHAR(50) AFTER apellido1;
```

Después de ejecutar todas las sentencias la tabla quedaría así:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL, apellido1
  VARCHAR(50) NOT NULL,
  apellido2 VARCHAR(50),
  sexo ENUM('H', 'M') NOT NULL DEFAULT 'M',
  fecha_nacimiento DATE NOT NULL
);
```

Manipulación de Tablas

Ejemplo de ALTER TABLE <tbl_name> DROP

DROP nos permite eliminar una columna de la tabla.
Suponemos que tenemos la siguiente tabla creada

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  apellido1 VARCHAR(50) NOT NULL,
  apellido2 VARCHAR(50),
  sexo ENUM('H', 'M') NOT NULL DEFAULT 'M',
  fecha_nacimiento DATE NOT NULL
);
```

Y queremos eliminar la columna fecha_nacimiento. En este caso usaremos la sentencia:

```
ALTER TABLE usuario DROP fecha_nacimiento;
```

Después de ejecutar esta sentencia la tabla quedaría así:

```
CREATE TABLE usuario (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  apellido1 VARCHAR(50) NOT NULL,
  apellido2 VARCHAR(50),
  sexo ENUM('H', 'M') NOT NULL DEFAULT 'M'
);
```

Lenguaje de Definición de Datos

Manipulación de Tablas

Consultar el listado de tablas disponibles

```
SHOW TABLES;
```

Muestra un listado de todas las tablas que existen en la base de datos con la que estamos trabajando.

Mostrar información sobre la estructura de una tabla

```
DESCRIBE nombre_tabla;
```

También podemos utilizar:

```
DESC nombre_tabla;
```

Esta sentencia se utiliza para mostrar información sobre la estructura de una tabla.

Mostrar la sentencia SQL de creación de una tabla

```
SHOW CREATE TABLE nombre_tabla;
```

Se puede utilizar para visualizar la sentencia SQL que sería necesaria ejecutar para crear la tabla que le estamos indicando como parámetro.